

Step-by-Step Lakehouse Platform Migration Guide

Follow this guide to set up the Data Lakehouse platform on your new laptop using the code stored in your GitHub repository.

Step 1: Install System Prerequisites

Before running the code, you need to install the required Java and Python environments. You can install them directly from your PowerShell terminal using the Windows Package Manager (winget).

1. Install Python 3.11

Open PowerShell on your new laptop and execute:

```
winget install --id Python.Python.3.11 -e
```

2. Install Java JDK 17

Open JDK is required by PySpark to run Spark SQL tasks. Run:

```
winget install --id Microsoft.OpenJDK.17 -e
```

3. Verify Installations

Close and reopen your PowerShell terminal, then execute:

```
python --version  
java -version
```

Confirm that Python 3.11 and Java 17 versions print out successfully.

Step 2: Clone the Project from GitHub

Open your PowerShell terminal on the new laptop, navigate to the folder where you want the project to live, and run:

```
git clone https://github.com/jpreddy-881/Project1.git  
cd Project1
```

Step 3: Install Python Libraries

Install PySpark and other core delta utilities needed to run the engine. Execute this inside the project root terminal:

```
pip install pyspark==3.5.0
```

Step 4: Locate and Update Hardcoded Path Values

Because data engines require absolute paths to read Delta files and metadata safely, we must update the paths referencing the old user directory (c:/Users/Dell/Downloads/project1) to match the new location on your laptop.

1. Update [config.yaml](file:///c:/Users/Dell/Downloads/project1/config/config.yaml)

Open the file located in config/config.yaml and replace c:/Users/Dell/Downloads/project1 with the new folder path.

- *Use forward slashes (/) even on Windows to prevent parsing errors.*

For example, if your project folder is cloned to D:/DataProjects/Project1, change the file content to match:

```
# Lakehouse Platform Configurations
dev:
  warehouse_dir: "D:/DataProjects/Project1/data/warehouse"
  metadata_path: "D:/DataProjects/Project1/data/metadata"
  log_file: "D:/DataProjects/Project1/data/logs/lakehouse.log"
  spark_packages: "io.delta:delta-spark_4.1_2.13:4.1.0"
  derby_home: "D:/DataProjects/Project1/data/metadata/.derby"

test:
  warehouse_dir: "D:/DataProjects/Project1/data/warehouse_test"
  metadata_path: "D:/DataProjects/Project1/data/metadata_test"
  log_file: "D:/DataProjects/Project1/data/logs/lakehouse_test.log"
  spark_packages: "io.delta:delta-spark_4.1_2.13:4.1.0"
  derby_home: "D:/DataProjects/Project1/data/metadata_test/.derby"

prod:
  warehouse_dir: "D:/DataProjects/Project1/data/warehouse_prod"
  metadata_path: "D:/DataProjects/Project1/data/metadata_prod"
  log_file: "D:/DataProjects/Project1/data/logs/lakehouse_prod.log"
  spark_packages: "io.delta:delta-spark_4.1_2.13:4.1.0"
  derby_home: "D:/DataProjects/Project1/data/metadata_prod/.derby"
```

2. Update [bootstrap_config.json](file:///c:/Users/Dell/Downloads/project1/config/bootstrap_config.json)

This bootstrap file tells the metadata loader where the Delta catalog resides. Update the path:

```
{"metadata_path": "D:/DataProjects/Project1/data/metadata_test"}
```

Step 5: Execute and Test

After updating your path values, run the main testing simulation in the project folder terminal:

```
python run_pipeline.py
```

This verification script will:

1. Initialize the Derby database metastores.
2. Bootstrap and seed all required Delta configurations and data quality rules tables under `data/metadata_test`.
3. Create mock landing data batches.
4. Execute `run_pipeline` incrementally.
5. Print conformed data frames and quality logs in stdout, confirming the setup succeeded.